Лабораторная работа №3

Проектирование схемы базы данных с помощью языковой модели

Дисциплина: Автоматизация научных исследований

Выполнил: Фролов Иван, группа 5040102/60201

Формат: индивидуально. Модель: Grok 4.7. Оба запроса в одном диалоге. Редактор: dbdiagram.io.

1. Выбранное задание

Вариант 20. Условие сохранено дословно в task/uslovie.txt. Переформулировки не было.

```
20. Станция скорой помощи

Персонал станции - список фельдшеров, медсестер/медбратьев, водителей. Есть парк машин минимум
двух типов (реанимация/обычная). На станции принимают вызовы (симптомы, фио, адрес пациента).
Есть несколько дежурных бригад. Бригада обслуживает «вызов» 20 минут. Потом может получить
новый. Любой сотрудник станции может заболеть/уволиться.

Требуется:
• Прием/увольнение/отметка о болезни сотрудника станции;
• Оформление вызова;
• Назначение бригады на вызов;
• Оформление отработки вызова (оказано лечение/умер/везем в больницу; если везут в больницу –
еще 20 минут на этот вызов);
• Оформление новой/списание старой машины;
• Составление графика работы бригад;

Отчеты:
• Статистика по вызовам за год;
• Бригады на выезде в текущий момент времени;
• Самые успешные/неуспешные бригады.
```

2. Первоначальный запрос и ответ

Начальный запрос: prompts/prompt1.txt: полный DBML для dbdiagram.io, без SQL и кода приложения, с явными допущениями в комментариях. Сырой ответ модели сохранён до правок в answers/grok-first.txt.

Модель предложила 8 таблиц и три перечисления.

Таблица	Назначение
employees	Сотрудники: роль, приём, увольнение без удаления строки
sick_leaves	Отметки о болезни и выздоровлении
vehicles	Парк машин двух типов, ввод и списание
crews	Постоянные дежурные бригады
crew_shifts, crew_shift_members	График смен: машина и состав
calls	Приём вызова: симптомы, ФИО, адрес
call_assignments	Назначение бригады и отработка; окно 20/40 минут

Явные допущения первой версии: успех: лечение или госпитализация, неуспех: смерть; на выезде считается по started_at … busy_until; годовая статистика по calls.received_at.

3. Первая диаграмма и проверка DBML

Исходный DBML вставлен в dbdiagram.io. Диаграмма построена без ошибок. Предупреждений редактора не было, отдельный скриншот warnings не сохранялся.

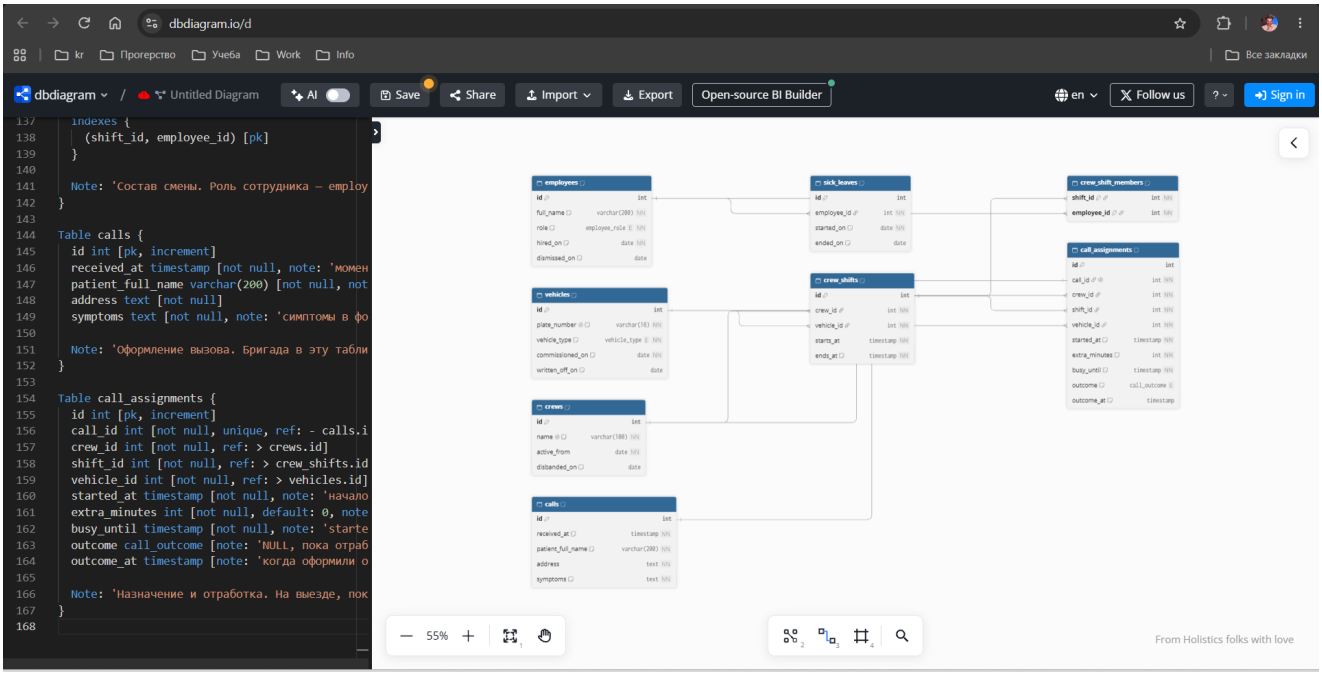


Рисунок 1. Первоначальная схема базы данных

4. Качественный разбор первой версии

Основные сущности для шести операций и трёх отчётов присутствуют. Проверка сосредоточена на согласованности длительности с исходом, связности смены с бригадой и машиной, простых CHECK и сохранности истории.

Проблема и место	Жизненная ситуация	Последствие	Что изменить
extra_minutes и busy_until не связаны с outcome	Исход оказано лечение, но extra_minutes = 20	Бригада на выезде дольше правила	CHECK: extra_minutes 0/20 и согласован с исходом
Пара outcome / outcome_at	Исход заполнен, время пустое	Нельзя датировать отработку	CHECK: оба NULL или оба заполнены
crew_id, shift_id и vehicle_id независимы	Смена бригады Б у бригады А	График и факт выезда расходятся	Составные FK к смене
Даты только в комментариях	ends_at ≤ starts_at; увольнение раньше приёма	Ключи такое не запрещают	CHECK одной строки; остальное к триггерам
История может быть стёрта	После отработки сменили crew_id или удалили машину	Статистика и рейтинг ломаются	Мягкое списание датами; запрет правок снаружи
Критерий успешности не задан	Нужен отчёт по бригадам	Рейтинг двусмыслен	Зафиксировать критерий из первой версии

5. Исправление и повторная проверка

Замечания переданы той же модели в том же диалоге (prompts/prompt2.txt). Исправленный DBML: answers/grok-second.txt. После вставки в dbdiagram.io диаграмма снова построена без ошибок и предупреждений.

Замечание	Изменение во второй версии	Результат
Длительность и исход	CHECK на extra_minutes, связь с outcome, busy_until через interval	Исправлено в DBML
outcome / outcome_at	CHECK пары и порядка; назначение без исхода допустимо	Исправлено в DBML
Бригада, смена, машина	Составные FK; машина выезда = машина смены	Исправлено в DBML
Даты и интервалы	CHECK на смены, больничные, увольнение, списание	Однострочные в DBML; межтабличные к триггерам
История	Мягкое завершение; T8-T11 запрещают DELETE и перепись	Уточнено; нужна внешняя реализация
Рейтинг бригад	Явный критерий успеха/неуспеха	Зафиксировано допущением

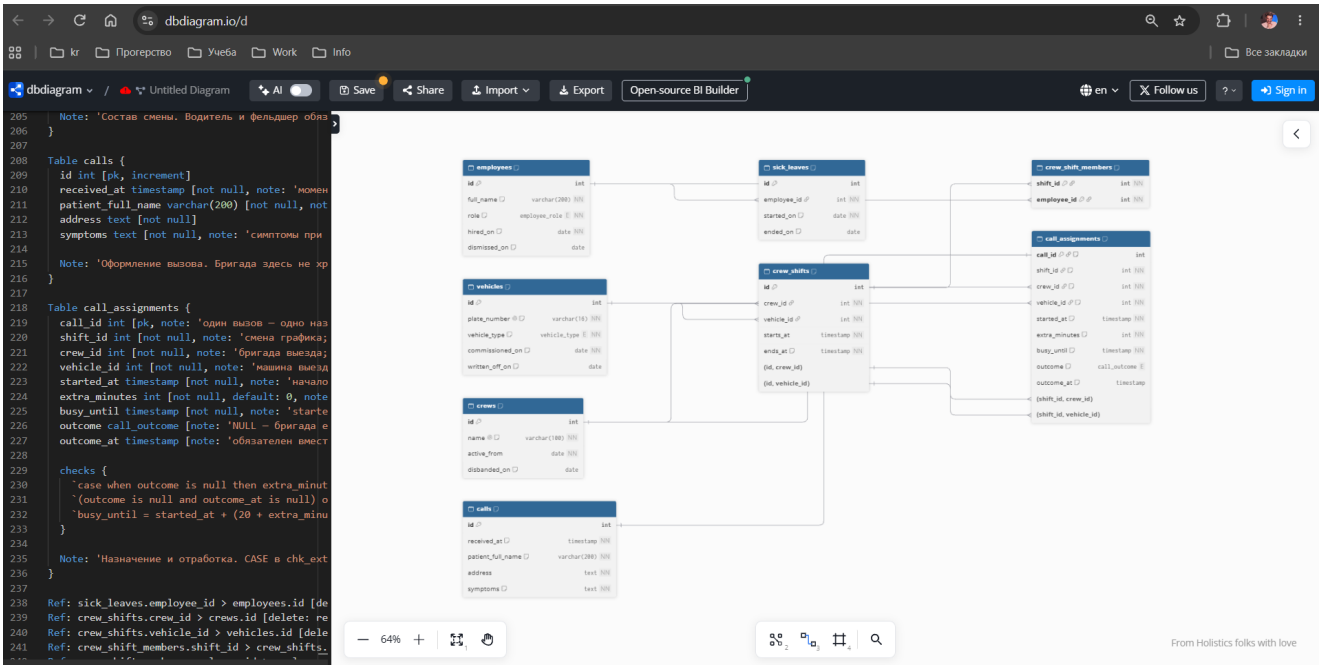


Рисунок 2. Исправленная схема базы данных

Повторная проверка ситуаций:

Ситуация	Результат анализа второй версии
Госпитализация	extra_minutes = 20 и окно 40 минут задаются CHECK
Лечение или смерть	extra_minutes = 0; окно 20 минут
Бригада ещё на выезде	outcome и outcome_at NULL допустимы
Смена другой бригады в назначении	Составной FK отклоняет запись
Увольнение / списание / расформирование	Строки остаются; прошлые вызовы читаются
Пересечение смен или окон выезда	Описано триггерами T3-T5, не обычным CHECK

6. Реализуемость отчётов

Отчёт	Данные и способ
Статистика по вызовам за год	Год calls.received_at; исход, бригада и тип машины из назначения и vehicles

Отчёт	Данные и способ
Бригады на выезде сейчас	started_at ≤ сейчас < busy_until; дежурство по графику сюда не входит
Самые успешные/неуспешные бригады	Доля успехов/неуспехов среди назначений с outcome; период по received_at

7. Вывод

Grok 4.7 в первой версии уже покрыл персонал, машины, бригады, график, вызовы и окно 20/40 минут. Качественный разбор выявил отсутствие строгой связи длительности с исходом, риск рассинхрона смены с бригадой/машиной, слабые CHECK дат и незащищённость истории. После замечаний модель добавила checks, составные внешние ключи и явно отделила межтабличные правила. Вторая диаграмма собрана без ошибок и предупреждений. Схема пригодна как основа учёта станции при соблюдении допущений и реализации триггеров T1-T11. Визуально корректная диаграмма всё равно нуждается в содержательной проверке: улучшение второй версии связано главным образом с ограничениями и поведением в конкретных ситуациях.

Приложение А. Первоначальный запрос

```
По приведённому ниже заданию спроектируй реляционную схему базы данных.

Выдай один полный блок DBML для dbdiagram.io:
таблицы, поля, типы данных, первичные и внешние ключи, связи.

Все операции и отчёты из условия должны быть реализуемы
на основе схемы. SQL-запросы и код приложения не нужны.
Необходимые допущения явно укажи в комментариях DBML.

Полное условие:
20. Станция скорой помощи

Персонал станции - список фельдшеров, медсестер/медбратьев, водителей. Есть парк машин минимум двух
типов (реанимация/обычная). На станции принимают вызовы (симптомы, фио, адрес пациента). Есть
несколько дежурных бригад. Бригада обслуживает «вызов» 20 минут. Потом может получить новый. Любой
сотрудник станции может заболеть/уволиться.

Требуется:
• Прием/увольнение/отметка о болезни сотрудника станции;
• Оформление вызова;
• Назначение бригады на вызов;
• Оформление отработки вызова (оказано лечение/умер/везем в больницу; если везут в больницу – еще
20 минут на этот вызов);
• Оформление новой/списание старой машины;
• Составление графика работы бригад;

Отчеты:
• Статистика по вызовам за год;
• Бригады на выезде в текущий момент времени;
• Самые успешные/неуспешные бригады.
```

Полный сырой ответ модели: answers/grok-first.txt.

Приложение Б. Запрос с замечаниями

```
Проверь и исправь схему Станция скорой помощи по замечаниям ниже.

Диаграмма в dbdiagram.io собралась без ошибок и предупреждений.

1. Длительность выезда и исход.
Сейчас можно поставить treatment_provided при extra_minutes = 20 или transported_to_hospital при
extra_minutes = 0. Тогда отчёт о бригадах на выезде врёт. Добавь checks: extra_minutes только 0 или
20, для госпитализации 20, для лечения и смерти 0. busy_until должен совпадать со started_at + 20 +
extra. Если interval в CHECK DBML не умеет, напиши что это триггер, не выдавай комментарий за
ограничение.

2. outcome и outcome_at.
Нельзя чтобы исход был, а время пустое, или outcome_at раньше started_at. Добавь check на пару и
порядок. Назначение без исхода оставь допустимым, бригада ещё на выезде.

3. Бригада, смена и машина в назначении.
Сейчас crew_id может быть от одной бригады, а shift_id от другой, и vehicle_id назначения может не
совпадать со сменой. Сделай так чтобы смена принадлежала той же бригаде, и зафиксируй откуда машина
выезда. Если составной FK неудобен, опиши как триггер. Не держи один факт в двух местах без правила.

4. Даты и интервалы.
Сейчас ends_at <= starts_at, ended_on < started_on, dismissed_on < hired_on и т.п. ключи не
запрещают. Добавь checks одной строки где можно. Межтабличное (смена покрывает started_at, окна не
пересекаются, в смене есть водитель и фельдшер, на смену не ставят уволенного или болеющего) отнеси
к триггерам.

5. История.
После отработки можно сменить crew_id или удалить машину, и статистика ломается. Оставь мягкое
увольнение и списание через даты, строки не удалять. Запрет менять поля после outcome если DBML не
умеет опиши снаружи.

6. Успешные и неуспешные бригады.
В условии критерий не задан. Зафиксируй как в первой версии: успех treatment_provided и
transported_to_hospital, неуспех deceased, рейтинг по заполненным outcome, период по calls.
received_at.

Удачные решения первой версии не ломай: отдельные таблицы, вызов без обязательного назначения, одно
назначение на вызов, окно через started_at/extra_minutes/busy_until, мягкое увольнение и списание,
enum ролей машин и исходов. Лишних сущностей не добавляй.

Исходные требования сохрани. Верни один полный блок DBML для dbdiagram.io. Допущения и правила под
триггеры пиши в комментариях. SQL не нужен.
```

Файл: prompts/prompt2.txt. Исправленный DBML: answers/grok-second.txt.